

BÁO CÁO KẾT QUẢ THỰC TẬP

Tuần 1: Làm quen với Backend cơ bản và NestJS

I. LÝ THUYẾT

1. Backend và Frontend

- Backend:** Là phần cốt lõi nằm bên dưới hệ thống, chịu trách nhiệm xử lý logic nghiệp vụ, giao tiếp với cơ sở dữ liệu, quản lý xác thực/phân quyền và cung cấp các dịch vụ API cho các ứng dụng khác. Người dùng cuối không thể nhìn thấy hoặc tương tác trực tiếp với Backend.
- Frontend:** Là giao diện đồ họa người dùng trực quan (UI/UX) được hiển thị trên trình duyệt hoặc ứng dụng di động (HTML, CSS, JavaScript), chịu trách nhiệm tiếp nhận tương tác từ người dùng và gửi yêu cầu (Request) tới Backend để lấy dữ liệu hiển thị.
- Bảng so sánh tóm tắt:**

Đặc điểm	Frontend	Backend
Môi trường chạy	Trình duyệt của người dùng	Máy chủ
Công nghệ phổ biến	React, Vue, Angular, HTML/CSS	NestJS, Express, Spring Boot, Go, Python
Nhiệm vụ chính	Hiển thị giao diện, trải nghiệm người dùng	Xử lý logic, bảo mật dữ liệu, tính toán

2. HTTP Request và Response

Giao tiếp giữa Frontend và Backend theo mô hình Client-Server thông qua giao thức HTTP:

- HTTP Request (Yêu cầu):** Gói tin do Client gửi lên Server, bao gồm:
 - URL/Endpoint:* Địa chỉ đường dẫn API (Ví dụ: /users).
 - HTTP Methods:* Phương thức hành động (**GET** để lấy, **POST** để tạo, **PUT** để cập nhật toàn bộ, **DELETE** để xóa).
 - Headers:* Chứa thông tin bổ sung (Ví dụ: Content-Type: application/json).
 - Body:* Dữ liệu định dạng JSON gửi kèm (thường dùng trong **POST** và **PUT**).
- HTTP Response (Phản hồi):** Gói tin do Server xử lý và trả về cho Client, bao gồm:
 - HTTP Status Code:* Mã trạng thái phản hồi (**200 OK** - Thành công, **201 Created** - Tạo mới thành công, **404 Not Found** - Không tìm thấy, **500 Internal Server Error** - Lỗi máy chủ).
 - Body:* Dữ liệu kết quả dạng JSON (Ví dụ: { "message": "Hello NestJS!" }).

3. Cách tổ chức dự án trên NestJS

- **Cách tổ chức dự án:**
 - **Controller:** Tiếp nhận các HTTP Request từ client, phân tích route/param và chuyển giao công việc cho lớp Service xử lý, sau đó trả về HTTP Response.
 - **Service (Provider):** Nơi tập trung toàn bộ logic nghiệp vụ của ứng dụng (CRUD dữ liệu, tính toán, xử lý logic).
 - **Module:** Đóng gói toàn bộ cấu phần liên quan (Controller, Service) thành một khối chức năng logic độc lập để tái sử dụng và quản lý hệ thống.

II. NỘI DUNG THỰC HÀNH & CÁC CÂU LỆNH CẤU HÌNH

1. Nội dung thực hành

Toàn bộ code được đẩy lên github trong folder week1

2. Câu lệnh cấu hình và chạy dự án

- **Khởi tạo dự án ban đầu bằng Nest CLI:**

```
npx @nestjs/cli new my-app
```

- **Di chuyển vào thư mục dự án:**

```
cd my-app
```

- **Tạo các cấu phần quản lý Users (Module, Controller, Service):**

```
npx @nestjs/cli g module users  
npx @nestjs/cli g controller users  
npx @nestjs/cli g service users
```

- **Cài đặt định nghĩa kiểu dữ liệu cho môi trường kiểm thử (Fix lỗi ẩn của Jest):**

```
npm install --save-dev @types/jest
```

- **Chạy toàn bộ các bài kiểm thử tự động (Unit Test) của dự án:**

```
npm run test
```

- **Khởi động máy chủ phát triển ở chế độ theo dõi thay đổi file:**

```
npm run start:dev
```

III. KẾT QUẢ KIỂM THỬ

Hệ thống đã triển khai đầy đủ và kiểm thử các endpoint theo đặc tả yêu cầu bằng công cụ Postman. Dữ liệu được lưu trữ trong một mảng (User[]) ở bộ nhớ đệm (In-memory Array) kèm theo bộ đếm ID tự tăng.

1. Kết quả chạy Unit Test trên Terminal

Toàn bộ các file kiểm thử .spec.ts của ứng dụng:

```
PS D:\GitHub\YootexIoT_Intern\week1\my-app> npm run test

> my-app@0.0.1 test
> jest

PASS src/users/users.service.spec.ts
PASS src/users/users.controller.spec.ts
PASS src/app.controller.spec.ts

Test Suites: 3 passed, 3 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        2.585 s
Ran all test suites.
PS D:\GitHub\YootexIoT_Intern\week1\my-app> |
```

```
C:\WINDOWS\system32\cmd. x + v
[11:51:12 AM] Starting compilation in watch mode...
[11:51:16 AM] Found 0 errors. Watching for file changes.

[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [NestFactory] Starting Nest application...
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [InstanceLoader] AppModule dependencies initialized +12ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [InstanceLoader] UsersModule dependencies initialized +0ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RoutesResolver] AppController {/}: +4ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/hello, GET} route +4ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RoutesResolver] UsersController {/users}: +0ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/users, POST} route +1ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/users, GET} route +0ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/users/:id, GET} route +1ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/users/:id, PUT} route +0ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [RouterExplorer] Mapped {/users/:id, DELETE} route +1ms
[Nest] 44452 - 07/28/2026, 11:51:17 AM LOG [NestApplication] Nest application successfully started +2ms
```

2. API Health: GET /hello

- **Mô tả:** Kiểm tra hệ thống, trả về một đối tượng JSON thông điệp chào mừng.
- **Kết quả phản hồi:** **200 OK** - { "message": "Hello NestJS!" }.

The screenshot displays the Postman interface with a workspace named "Hoàng Dương Minh's Workspace". The left sidebar shows a collection of API requests under "My App - NestJS (In-Memory Array)", including "Health" and "Users". The "GET /hello" request is selected. The main panel shows the request details, including the method "GET", the URL "((baseUri)) /hello", and the "Send" button. Below the request details, the "Body" tab is active, showing the response in JSON format:

```
{  "message": "Hello NestJS!"}
```

. The status bar at the bottom indicates a successful response with "200 OK", a response time of "15 ms", and a size of "262 B".

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

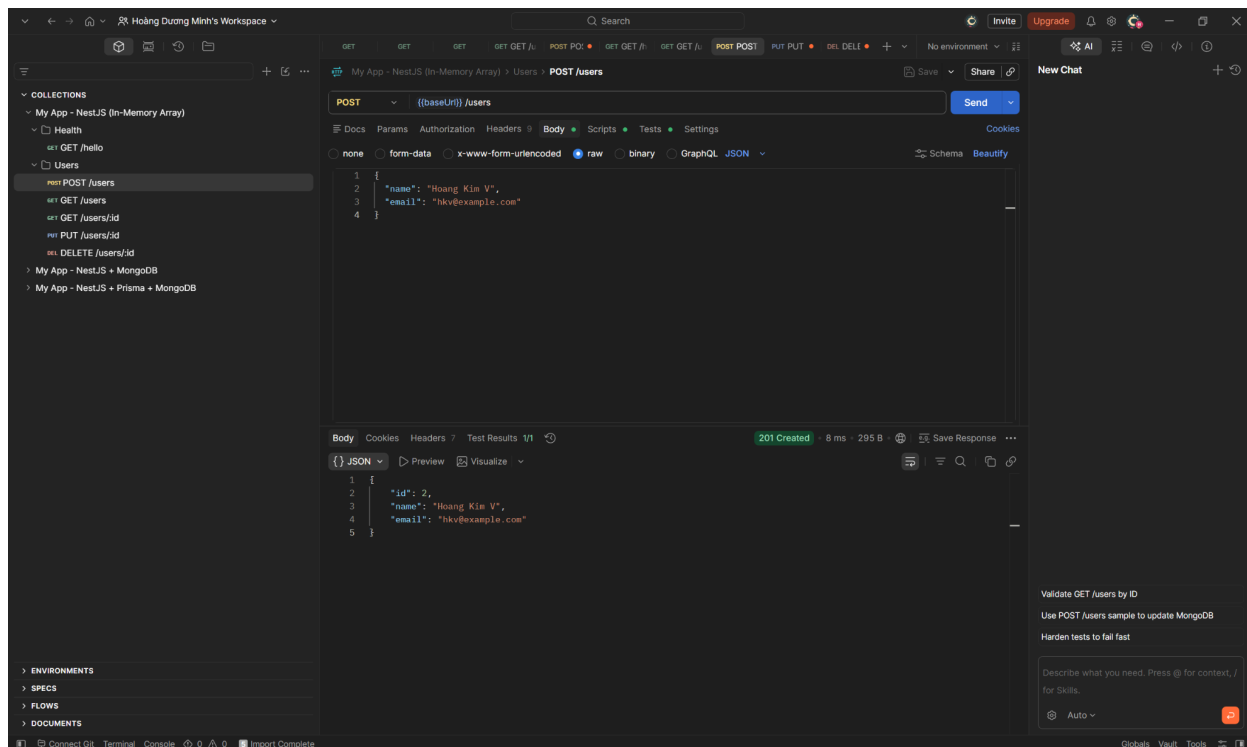
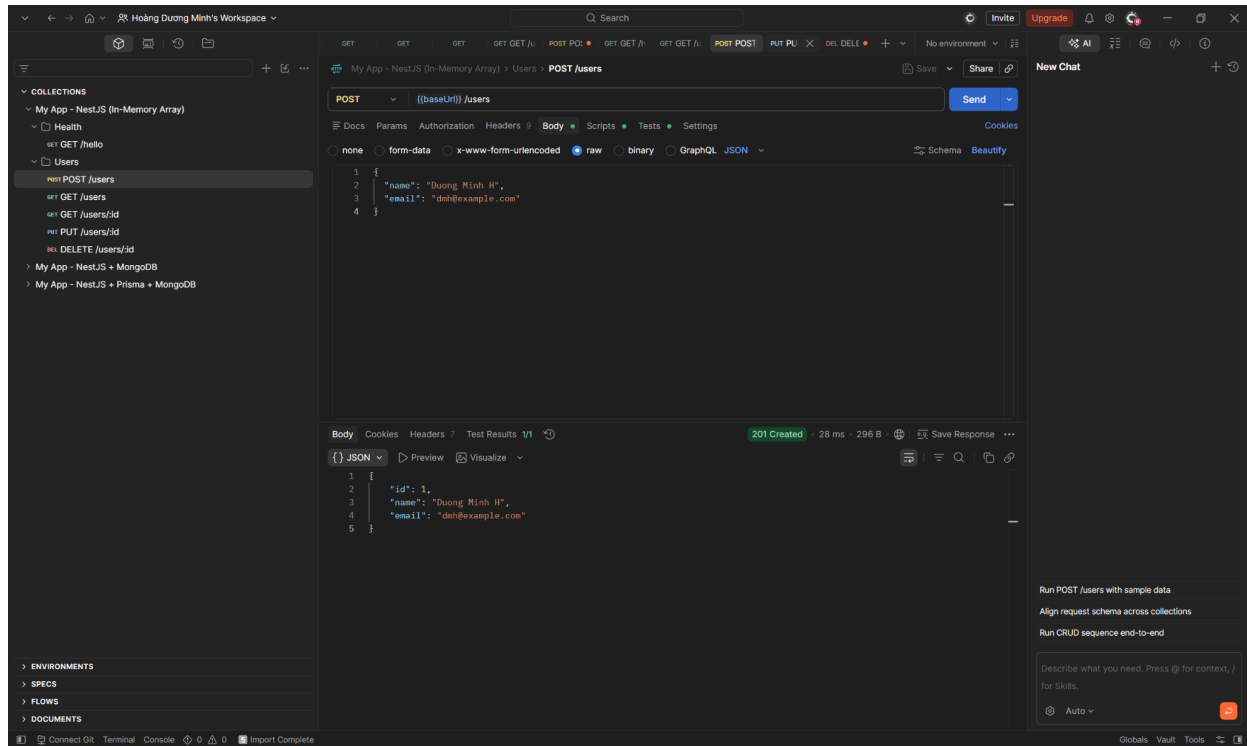
Body

200 OK - 15 ms - 262 B

```
{  "message": "Hello NestJS!"}
```

3. API CRUD Users: POST /users

- **Mô tả:** Tạo một người dùng mới với các thuộc tính name và email, tự động sinh ID tăng dần khởi đầu từ 1.
- **Kết quả phản hồi:** 201 Created kèm thông tin User mới được đẩy vào mảng.



4. API CRUD Users: GET /users

- **Mô tả:** Lấy ra toàn bộ danh sách các đối tượng người dùng hiện đang được lưu giữ tạm thời trong mảng.
- **Kết quả phản hồi:** **200 OK** trả về một mảng danh sách người dùng.

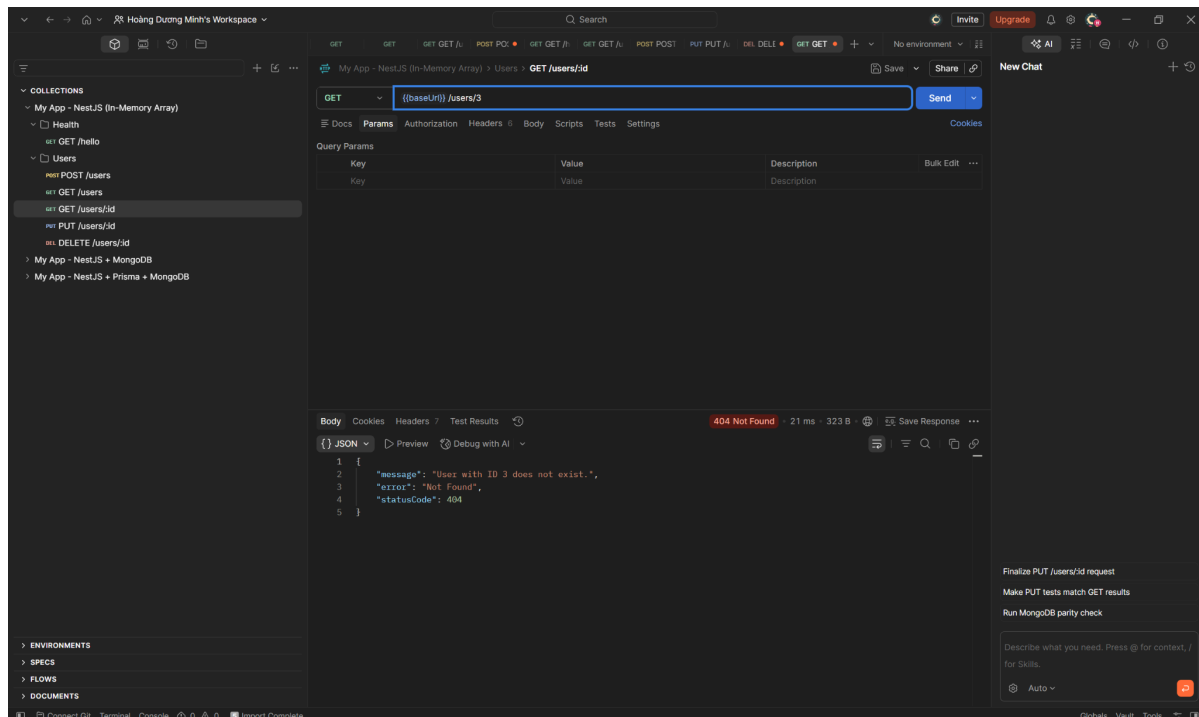
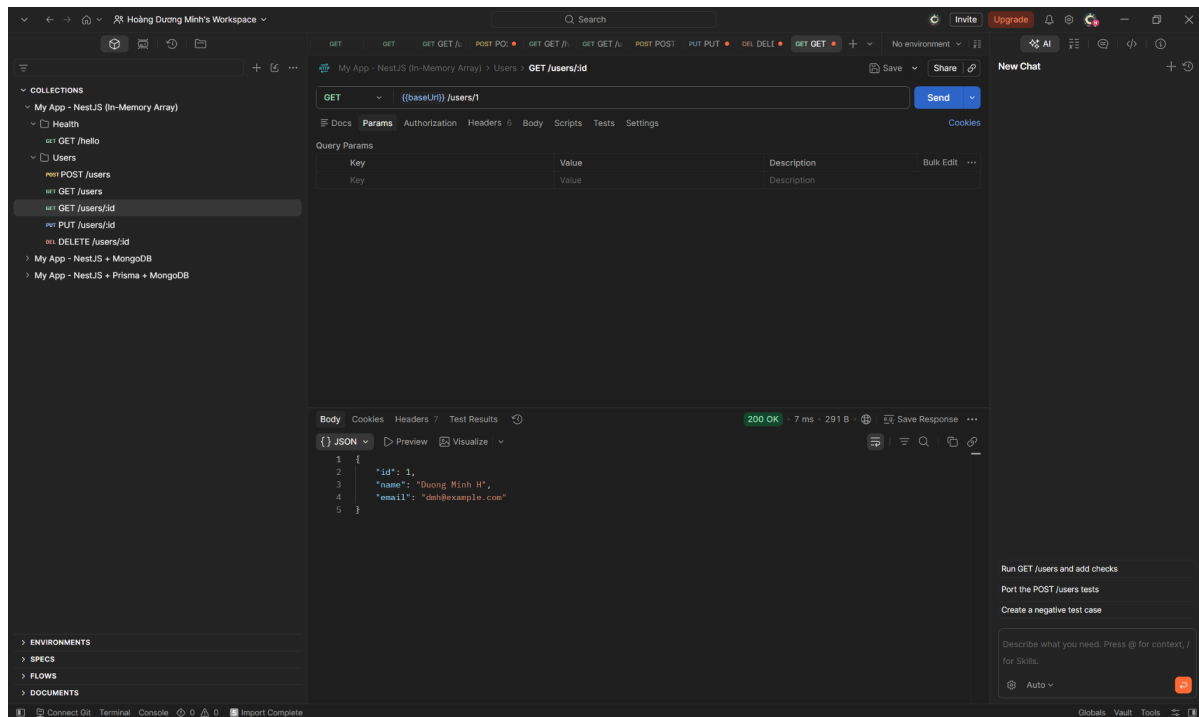
The screenshot displays the Postman interface for a REST client. The left sidebar shows a collection named 'My App - NestJS (In-Memory Array)' with a sub-collection 'Users'. The 'GET /users' endpoint is selected. The main panel shows the 'Params' tab with a single parameter 'Key' and 'Value'. The 'Body' tab is active, showing a JSON response with a status of '200 OK' and a response time of '11 ms'. The JSON response is as follows:

```
1 {
2   {
3     "id": 1,
4     "name": "Duong Minh H",
5     "email": "dmh@example.com"
6   },
7   {
8     "id": 2,
9     "name": "Hoang Kim V",
10    "email": "hkv@example.com"
11  }
12 }
```

The right sidebar contains a 'New Chat' button and a chat interface with a text input and a 'Send' button.

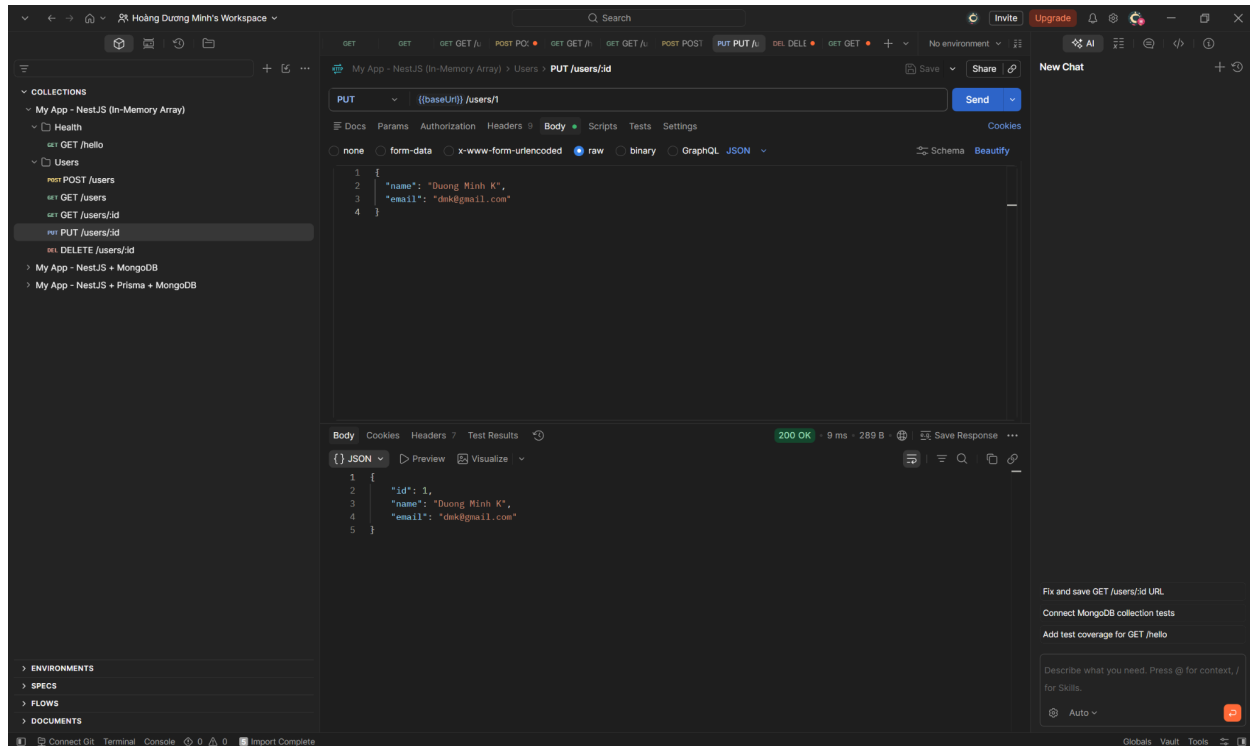
5. API CRUD Users: GET /users/:id

- **Mô tả:** Xem chi tiết thông tin của một người dùng dựa theo mã số định danh ID truyền vào.
- **Kết quả phản hồi:** **200 OK** nếu ID hợp lệ hoặc **404 Not Found** nếu ID không tồn tại trong hệ thống.



6. API CRUD Users: PUT /users/:id

- **Mô tả:** Cập nhật thông tin các thuộc tính (name, email) của người dùng tương ứng với ID chỉ định.
- **Kết quả phản hồi:** **200 OK** trả về thông tin đối tượng sau khi đã sửa đổi thành công.



7. API CRUD Users: DELETE /users/:id

- **Mô tả:** Tìm kiếm và xóa hoàn toàn đối tượng người dùng ra khỏi mảng lưu trữ dựa vào ID.
- **Kết quả phản hồi:** 200 OK kèm thông báo chuỗi xác nhận xóa thành công.

The screenshot displays a REST client interface with a sidebar on the left showing a collection of endpoints. The main panel shows a DELETE request to the endpoint `/users/1`. The response is a 200 OK status with a response time of 6 ms and a body containing a success message.

Endpoint: DELETE `/users/1`

Response: 200 OK - 6 ms - 284 B

Body (JSON):

```
1 {
2   "message": "Successfully deleted user with ID 1"
3 }
```

Left Sidebar (Collections):

- My App - NestJS (In-Memory Array)
 - Health
 - GET /hello
 - Users
 - POST /users
 - GET /users
 - GET /users/:id
 - PUT /users/:id
 - DELETE /users/:id
- My App - NestJS + MongoDB
- My App - NestJS + Prisma + MongoDB

Bottom Panel (Environments, Specs, Flows, Documents):

- ENVIRONMENTS
- SPECS
- FLows
- DOCUMENTS

Right Panel (New Chat):

Finalize PUT /users/:id request
Make PUT tests match GET results
Run MongoDB parity check

Describe what you need. Press @ for context, / for Skills.

Auto